

AWS DevOps

Complete Interview Preparation Guide

IAM · EC2 · VPC · S3 · Lambda · ECS · EKS · CodePipeline · CloudWatch · and more

Compiled from hands-on project notes
2024 – 2025

1. IAM — Identity and Access Management

IAM is the gatekeeper for your entire AWS account. It controls who can do what with which AWS resources.

Core Concepts

Term	Definition
Authentication	Verifying who you are (username/password, access keys, MFA)
Authorization	What actions you are allowed to perform after authentication
User	An individual identity with long-term credentials (username + password or access keys)
Group	A collection of IAM users sharing the same permissions — assign policies to groups, not individual users
Role	A temporary identity with specific permissions. Assumed by users, apps, or AWS services (e.g., EC2 accessing S3)
Policy	A JSON document defining allowed/denied actions on AWS resources. Attached to users, groups, or roles

How Policies Work

Policies are JSON documents. Example — allow S3 read only:

```
{ "Effect": "Allow", "Action": ["s3:GetObject"], "Resource": "arn:aws:s3:::my-bucket/*" }
```

- Effect: Allow or Deny
- Action: Which API calls are permitted (e.g., s3:GetObject, ec2:StartInstances)
- Resource: Which specific resource (ARN) the rule applies to
- Principal: Who the policy applies to (used in resource-based policies)

Best Practices

- Never use root account for daily tasks — create IAM users instead
- Always enable MFA on root and admin accounts
- Assign permissions to Groups, then add users to groups
- Use Roles for service-to-service access (e.g., Lambda accessing DynamoDB)
- Follow least privilege principle — grant only what is needed
- Rotate access keys regularly

□ **Key Point:** Interview tip: Know the difference between a Role (temporary, assumed) and a User (long-term credentials). Roles are used for EC2 instances, Lambda functions, and cross-account access.

2. EC2 — Elastic Compute Cloud

EC2 provides resizable virtual servers (instances) in the cloud. You choose the OS, CPU, RAM, storage, and networking.

Instance Type Families

Family	Use Case	Examples
General Purpose	Balanced CPU/memory — web servers, dev environments	t3.micro, m5.large
Compute Optimized	CPU-heavy workloads — batch processing, gaming servers	c5.large, c6g.xlarge
Memory Optimized	Large in-memory datasets — Redis, real-time analytics	r5.xlarge, x1e.32xlarge
Storage Optimized	High sequential I/O — data warehousing, Hadoop	i3.large, d2.8xlarge
Accelerated Computing	GPU/FPGA workloads — ML training, video rendering	p3.2xlarge, g4dn.xlarge

Accessing EC2

- SSH with PEM key — always set 600 permissions first:

```
chmod 600 my-key.pem
ssh -i my-key.pem ubuntu@<public-ip>
```

- If your app is not reachable in browser: add an inbound rule in the Security Group
 - Instance → Security → Security Group → Inbound Rules → Add Rule

Key Concepts

- Elastic IP: A static public IP that can be reassigned between instances
- EBS Volume: Block storage attached to an EC2 instance (like a hard drive)
- AMI (Amazon Machine Image): A snapshot/template used to launch instances
- User Data: Scripts that run automatically when an instance first starts
- Auto Scaling Group: Automatically adds/removes instances based on demand

EC2 Pricing Models

Model	Description	Best For
On-Demand	Pay per hour/second, no commitment	Variable workloads, testing
Reserved	1 or 3 year commitment, up to 75% cheaper	Stable predictable workloads
Spot	Bid on unused capacity, up to 90% cheaper — can be interrupted	Fault-tolerant batch jobs

Model	Description	Best For
Dedicated Host	Physical server dedicated to you	Compliance/licensing requirements

3. VPC — Virtual Private Cloud

A VPC is your own isolated network inside AWS. Think of it as your private data center in the cloud.

VPC Architecture — Full Flow

Here is how a request travels from the internet to your application:

Step	Component	Role
1	Internet Gateway (IGW)	Entry/exit point for the VPC — connects VPC to the internet
2	Public Subnet	Subnet with a route table pointing to the IGW — resources here have public IPs
3	Load Balancer (ALB)	Sits in public subnet, receives internet traffic and distributes it
4	Route Table	Directs traffic: 0.0.0.0/0 → IGW (for public), local routes for private
5	NACL	Subnet-level stateless firewall — evaluates rules top-down by rule number
6	Private Subnet	No direct internet route — used for EC2 instances, databases
7	Security Group	Instance-level stateful firewall — allows specific ports and protocols
8	Application	Your actual service running on EC2 in the private subnet
9	NAT Gateway	Allows private subnet instances to initiate outbound internet connections

CIDR and Subnets

- VPC CIDR example: 172.16.0.0/16 — gives 65,536 IP addresses
- Subnet CIDRs: 172.16.1.0/24 (256 IPs, 251 usable) per subnet
- Public subnet = has a route to IGW in its route table
- Private subnet = no IGW route; uses NAT Gateway for outbound access
- Availability Zones: Spread subnets across multiple AZs for high availability

Security Groups vs NACLs

Feature	Security Group	NACL
Level	Instance level	Subnet level
State	Stateful — response traffic is auto-allowed	Stateless — must explicitly allow both directions

Feature	Security Group	NACL
Rules	Allow only (no explicit deny)	Allow AND Deny
Rule Order	All rules evaluated together	Evaluated by rule number — lowest number first
Use Case	Fine-grained per-instance control	Block traffic at subnet boundary (e.g., block port 8000 company-wide)

VPC Project — Architecture

- Create VPC with 2 public + 2 private subnets across 2 AZs
- NAT Gateway in each AZ for private subnet outbound access
- Auto Scaling Group launches EC2 in private subnets
- Bastion Host in public subnet — SSH into private instances via it
- Application Load Balancer in public subnets — forwards to private EC2 instances
- SCP command to copy .pem key to bastion host:

```
scp -i aws3-login.pem aws3-login.pem ubuntu@<bastion-public-ip>:/home/ubuntu
```

□ **Key Point:** Interview tip: Know why private subnets don't have public IPs — it's a security best practice. The load balancer (public) is the only entry point from the internet.

4. S3 — Simple Storage Service

S3 is AWS's object storage service. It is globally accessible, infinitely scalable, and designed for 99.999999999% (11 nines) durability.

Key Features

- Objects (files) stored in Buckets
- Each object accessible via a unique URL
- Default encryption for all objects
- Versioning: Keep multiple versions of files — protects against accidental deletion
- Bucket policies: JSON-based, control who can access what
- Static website hosting: Serve HTML/CSS/JS directly from S3

Storage Classes

Class	Use Case	Retrieval
Standard	Frequently accessed data	Milliseconds
Standard-IA	Infrequently accessed, but needs fast retrieval	Milliseconds
One Zone-IA	Infrequent access, lower cost, single AZ	Milliseconds

Class	Use Case	Retrieval
Glacier Instant	Archive with instant retrieval	Milliseconds
Glacier Flexible	Archive — occasional access acceptable	Minutes to hours
Glacier Deep Archive	Long-term archive, rarely accessed	Hours
Intelligent-Tiering	Automatically moves data to right tier	Varies

Common Configurations

Block all public access except root account

```
{ "Effect": "Deny", "Principal": "*", "Action": "s3:*", "Condition": {
  "StringNotEquals": { "aws:PrincipalArn": "arn:aws:iam::ACCOUNT_ID:root" } } }
```

Allow public read (for static website)

```
{ "Effect": "Allow", "Principal": "*", "Action": "s3:GetObject", "Resource":
  "arn:aws:s3:::my-bucket/*" }
```

Static Website Hosting Setup

Steps: Properties → Static website hosting → Enable → set index.html → Permissions → unblock public access → add public read policy

5. Route 53 — DNS Service

Route 53 is AWS's managed DNS service. It translates domain names into IP addresses and routes traffic globally.

Three Main Capabilities

- Domain Registration: Register or transfer domains (like GoDaddy, but on AWS)
- Hosted Zones & DNS Records: Create A, CNAME, MX, TXT records to point domains to resources
- Health Checks: Monitor endpoints and route traffic away from unhealthy resources

Common Record Types

Record	Purpose
A	Maps domain to IPv4 address
AAAA	Maps domain to IPv6 address

Record	Purpose
CNAME	Alias for another domain (cannot be used at zone apex)
Alias	AWS-specific — maps to AWS resources like ALB, CloudFront (can be used at apex)
MX	Mail server routing
TXT	Text records — used for domain verification, SPF

Routing Policies

- Simple: Single resource, no health checks
- Weighted: Split traffic by percentage (e.g., 90%/10% for A/B testing or gradual rollouts)
- Latency-based: Route to region with lowest latency for the user
- Failover: Primary/secondary — automatically fails over to backup
- Geolocation: Route based on user's country or continent
- Multi-value: Return multiple healthy records

6. Security Groups & NACLs — Deep Dive

Security Groups

- Virtual firewall at the instance level
- Rules only allow — you cannot explicitly deny with SGs
- Stateful: If you allow inbound port 80, the response traffic is automatically allowed out
- Multiple SGs can be attached to one instance
- Changes take effect immediately
- Can reference other SGs as source (e.g., 'allow traffic from the load balancer SG')

NACLs

- Operate at subnet level — protect all instances in a subnet
- Stateless: Must explicitly allow both inbound AND outbound for two-way communication
- Rules evaluated in numeric order — lower number = higher priority
- Can explicitly DENY traffic (unlike SGs)
- Company policy example: Block port 8000 across all instances by adding NACL deny rule

□ **Note:** Navigation: VPC → Network ACLs → Select VPC → Inbound/Outbound Rules → Add Rule

□ **Key Point:** Interview trap: NACLs are stateless — if you add an inbound ALLOW for port 80, you also need an outbound ALLOW for the ephemeral response ports (1024-65535). Security Groups don't have this problem.

7. CloudWatch — Monitoring & Observability

CloudWatch is AWS's unified monitoring service. It collects metrics, logs, and events from virtually every AWS service.

Four Pillars

- Metrics: Numerical data over time — CPU utilization, request count, error rate
- Logs: Text output from applications and services — centralized in Log Groups
- Alarms: Trigger actions (SNS notification, Auto Scaling) when metrics breach thresholds
- Events/EventBridge: React to changes in AWS environment (e.g., EC2 state change)

Viewing Metrics

CloudWatch → Metrics → EC2 → Select instance → Choose metric (e.g., CPUUtilization)

Creating an Alarm

CloudWatch → Alarms → Create Alarm → Select metric → Set threshold → Choose action

- Action options: Send SNS notification (email/SMS), trigger Auto Scaling, stop/terminate instance
- SNS (Simple Notification Service): Sends alerts via email, SMS, or HTTP endpoints

Custom Metrics

If CloudWatch doesn't have the metric you need (e.g., memory usage, disk usage), you can push custom metrics using the CloudWatch agent or AWS SDK.

CPU Spike Simulation Script

Used to test CloudWatch alarms — this script spikes CPU for 30 seconds at 80%:

```
python3 spike_cpu.py # Runs a tight arithmetic loop to simulate high CPU
```

8. Lambda — Serverless Compute

Lambda is AWS's serverless function service. You upload code, define a trigger, and AWS handles everything else — no servers to provision or manage.

Key Characteristics

- Event-driven: Triggered by S3 uploads, API Gateway, CloudWatch Events, SQS, etc.
- Pay per invocation: Only charged when code is running (no idle cost)
- Scales automatically: Handles thousands of concurrent requests

- Max timeout: 15 minutes per invocation
- Supports Python, Node.js, Java, Go, Ruby, .NET, and custom runtimes

EBS Snapshot Cost Optimization — Lambda Project

This Lambda function deletes stale EBS snapshots to reduce storage costs. Logic:

- Get all EBS snapshots owned by the account
- Get all running EC2 instances
- For each snapshot: if it has no volume — delete it
- If it has a volume but the volume is not attached to any running instance — delete it
- If the volume no longer exists (instance was deleted) — delete the snapshot

Lambda Permissions

Lambda needs IAM permissions to call other AWS services. Attach policies to its execution role:

- AmazonEC2FullAccess — to describe and delete snapshots
- AWSLambdaBasicExecutionRole — to write logs to CloudWatch
- CloudWatchFullAccess — if publishing custom metrics

Lambda + Cron (Scheduled Execution)

Use CloudWatch Events (EventBridge) to run Lambda on a schedule — for example, run the snapshot cleanup every day at 2 AM:

```
Rate expression: rate(1 day) | Cron: cron(0 2 * * ? *)
```

9. ECR — Elastic Container Registry

ECR is AWS's fully managed Docker container registry. Think of it as a private Docker Hub that lives inside your AWS account.

ECR vs Docker Hub

Feature	ECR	Docker Hub
Default visibility	Private	Public
Integration	Native AWS (ECS, EKS, CodePipeline)	General purpose
Authentication	AWS IAM credentials	Docker credentials
Scanning	Built-in vulnerability scanning on push	Available on paid plans
Cost	Storage + data transfer charges	Free tier available

Push Commands

```
# Authenticate Docker to ECR
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin <account>.dkr.ecr.us-east-1.amazonaws.com
# Build, tag, and push
docker build -t my-app .
docker tag my-app:latest <account>.dkr.ecr.us-east-1.amazonaws.com/my-repo:latest
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/my-repo:latest
```

Best Practices

- Enable immutable tags: Prevents tag from being overwritten — ensures image traceability
- Enable push scan: Automatically scans for CVEs on every image push
- Use lifecycle policies: Automatically delete old/untagged images to reduce cost

10. ECS — Elastic Container Service

ECS is AWS's managed container orchestration service — similar to Kubernetes but AWS-native and simpler to operate.

ECS Components

Component	Description	Kubernetes Equivalent
Cluster	Logical grouping of infrastructure (Fargate or EC2)	Cluster
Task Definition	Blueprint: which container image, CPU, memory, ports, env vars	Pod spec
Task	A running instance of a Task Definition	Pod
Service	Maintains N running tasks, integrates with load balancer for HA	Deployment + Service

Fargate vs EC2 Launch Type

Aspect	Fargate	EC2
Infrastructure management	None — fully serverless	You manage EC2 instances
Scaling	Automatic	Requires Auto Scaling Group
Cost	Pay per vCPU/memory used	Pay for EC2 instances (even when idle)
Best for	Simplicity, variable workloads	Cost savings at scale, GPU workloads

ECS Limitations vs Kubernetes

- AWS-specific: Cannot run ECS workloads on other clouds
- No CRDs: Cannot extend ECS with custom resources like Kubernetes CRDs
- Less ecosystem: Fewer third-party integrations compared to Kubernetes

Deploying on ECS — Steps

1. Create ECR repo and push your Docker image
2. Create ECS Cluster (select Fargate)
3. Create Task Definition (image URL, port, CPU/memory, CloudWatch logging)
4. Deploy → Run Task (or create a Service for HA)
5. For HA with load balancing: Create a Service and attach an ALB

11. EKS — Elastic Kubernetes Service

EKS is AWS's managed Kubernetes service. It handles the control plane (API server, etcd, scheduler) while you manage the data plane (worker nodes).

EKS vs Self-Managed Kubernetes

Aspect	EKS	Self-Managed K8s
Control plane	AWS managed — HA by default	You manage — must configure HA yourself
Updates	AWS handles control plane upgrades	Manual upgrades
Integration	Native IAM, ALB, EBS, ECR integration	Requires manual setup
Cost	Control plane: ~\$0.10/hr + worker nodes	Only worker node cost

Worker Node Options

- Fargate: Fully serverless — AWS manages nodes entirely
- EC2 (Managed Node Group): AWS manages lifecycle but you pay for EC2; need ASG for HA
- EC2 (Self-managed): Full control, but you manage everything

OIDC and IRSA — IAM for Pods

By default, pods cannot access AWS services securely. IRSA (IAM Roles for Service Accounts) solves this:

- Configure OIDC provider for the EKS cluster

- Create an IAM Role with trust policy pointing to the Kubernetes service account
- Annotate the service account with the IAM role ARN
- Pods using that service account automatically get temporary AWS credentials

□ **Note:** This is far more secure than storing AWS access keys inside pods or container images.

Deploying the 2048 Game on EKS — Full Flow

1. Create Cluster

```
eksctl create cluster --name demo-cluster-1 --region us-east-1 --fargate
```

2. Update kubeconfig

```
aws eks update-kubeconfig --name demo-cluster-1 --region us-east-1
```

3. Create Fargate Profile for app namespace

```
eksctl create fargateprofile --cluster demo-cluster-1 --region us-east-1 --name alb-sample-app --namespace game-2048
```

4. Deploy app

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.5.4/docs/examples/2048/2048_full.yaml
```

5. Configure OIDC

```
eksctl utils associate-iam-oidc-provider --cluster demo-cluster-1 --approve
```

6. Install AWS Load Balancer Controller (via Helm)

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system --set clusterName=demo-cluster-1 ...
```

How Ingress Works on EKS

- You deploy an Ingress Controller (AWS Load Balancer Controller) as a pod
- The controller watches for Ingress resources using Kubernetes API
- When an Ingress resource is created, the controller automatically creates an ALB on AWS
- The ALB is in public subnets and routes traffic to pods
- `kubectl get ingress -A` shows the ALB DNS as the address

12. CloudFront — Content Delivery Network

CloudFront is AWS's CDN service. It caches content at 400+ edge locations worldwide to reduce latency for global users.

How CDN Works

Without CDN: A user in India requesting content stored in US-East must traverse the full internet path — high latency. With CloudFront: Content is cached at the nearest edge location (e.g., Mumbai). Subsequent requests are served from the edge — millisecond response.

CloudFront + S3 Static Website Setup

Steps to serve a private S3 bucket securely through CloudFront:

- Create S3 bucket, enable versioning, block ALL public access
- Upload index.html
- Enable static website hosting in bucket properties
- Create CloudFront distribution, select S3 bucket as origin
- In origin settings: Enable Legacy OAI (Origin Access Identity) — creates new OAI and updates bucket policy automatically
- Set Default Root Object to index.html
- Access site via CloudFront distribution domain (not S3 URL)

❑ **Note:** S3 bucket remains private. Only CloudFront (via OAI) can read from it. Direct S3 URL access is denied.

CloudFront Use Cases

- Static websites and SPAs
- Video streaming — adaptive bitrate delivery
- API acceleration — cache GET responses at edge
- DDoS protection via AWS Shield (integrated automatically)
- HTTPS termination at edge with ACM certificates

13. Load Balancers — ALB, NLB, GWLB

AWS provides three load balancer types, each operating at a different OSI layer.

OSI Model Quick Reference

Layer	Name	What it sees	AWS Service
7	Application	HTTP/HTTPS headers, cookies, URL paths	ALB
4	Transport	TCP/UDP ports and IP addresses only	NLB
3	Network	Raw IP packets	GWLB
3/2	Network/Data Link	IP addresses, MAC addresses	Router/Switch

Application Load Balancer (ALB)

- Operates at Layer 7 — understands HTTP/HTTPS
- Routes based on: URL path (/api/* vs /web/*), hostname, HTTP headers, query strings
- Supports: WebSockets, HTTP/2, gRPC

- Sticky sessions via cookies — keeps user on same target
- Best for: Microservices, web applications, REST APIs
- Use with ECS, EKS, EC2 target groups

□ **Note:** ALB terminates SSL — handles HTTPS decryption so your backend only sees HTTP

Network Load Balancer (NLB)

- Operates at Layer 4 — sees only TCP/UDP ports and IPs
- Extremely fast — handles millions of requests per second with ultra-low latency
- Preserves source IP address of clients
- Static IP address / Elastic IP support — useful for firewall whitelisting
- Sticky sessions based on source IP (not cookies)
- Best for: Gaming, financial trading, IoT, VoIP, custom TCP/UDP protocols
- Use when your backend needs a static IP or requires raw TCP connections

Gateway Load Balancer (GWLB)

- Operates at Layer 3 — IP packet level
- Used to insert third-party virtual appliances into traffic flow
- Traffic flow: Internet → GWLB → Firewall/IDS appliance → GWLB → Application
- Use cases: Centralized firewall, intrusion detection/prevention, deep packet inspection

Cookies — A Primer

Cookies are small text files stored in the browser. They solve HTTP's statelessness problem:

- Login sessions: Server saves session_id cookie → browser sends it on every request → server knows who you are
- Preferences: Language, theme, region stored client-side
- Session cookies: Deleted when browser closes
- Persistent cookies: Have an expiry date set by the server

□ **Key Point:** Stolen cookies = stolen session. Protect cookies with HttpOnly (no JS access) and Secure (HTTPS only) flags.

14. CloudFormation Templates (CFT)

CloudFormation lets you define AWS infrastructure as code (YAML or JSON). You describe what you want and AWS creates it — repeatable, version-controlled infrastructure.

Core Concepts

Term	Description
Template	YAML/JSON file describing the infrastructure

Term	Description
Stack	A deployed instance of a template — all resources created together
Change Set	Preview changes before applying them
Drift Detection	Detect when actual resource config differs from the template
Parameters	Input values — make templates reusable
Outputs	Export values for use by other stacks

Example — S3 Bucket with Versioning

```
Resources:
  S3Bucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: my-demo-bucket
      VersioningConfiguration:
        Status: Enabled
```

Drift Detection

If you manually delete an S3 bucket that was created by a CFT stack, drift detection will flag it:

Stack → Actions → Detect Drift → View Drift Results → Shows resource as DELETED/MODIFIED

VS Code Extensions

- YAML: Syntax highlighting and validation for CloudFormation YAML
- AWS Toolkit: Auto-complete for CFT resource types and properties

Infrastructure Composer

Visual drag-and-drop designer for CFT — go to CloudFormation → Stacks → Build from Infrastructure Composer. Drag resources from the left panel, switch to YAML view to edit properties.

15. CI/CD — CodePipeline, CodeBuild, CodeDeploy

AWS's native CI/CD suite automates the path from code commit to production deployment.

The Three Services

Service	Role	Equivalent Tool
CodeCommit	Managed Git repository — private source control on AWS	GitHub / GitLab

Service	Role	Equivalent Tool
CodeBuild	Managed build service — runs buildspec.yml to compile, test, and package code	Jenkins / GitHub Actions
CodeDeploy	Automated deployment to EC2, ECS, Lambda — runs appspec.yml scripts	Ansible / Spinnaker
CodePipeline	Orchestrates the full CI/CD pipeline — connects source → build → deploy	GitLab CI / Jenkins Pipeline

CodeBuild — buildspec.yml

The buildspec.yml defines build phases. For Docker-based Python project:

```
phases:
  install:      # Install runtimes
    runtime-versions:
      python: 3.11
  pre_build:    # Login to Docker, install dependencies
    commands:
      - echo $DOCKER_REGISTRY_PASSWORD | docker login -u $DOCKER_REGISTRY_USERNAME --password-stdin
  build:        # Build Docker image
    commands:
      - docker build -t $DOCKER_URL/$DOCKER_USERNAME/my-app:latest .
  post_build:   # Push image to registry
    commands:
      - docker push $DOCKER_URL/$DOCKER_USERNAME/my-app:latest
```

Storing Secrets for CodeBuild

- Store Docker credentials in Systems Manager Parameter Store as SecureString
- Reference in buildspec.yml under env.parameter-store
- CodeBuild IAM role needs AmazonSSMFullAccess

CodeDeploy — appspec.yml

appspec.yml defines lifecycle hooks — which scripts to run at each deployment stage:

```
hooks:
  ApplicationStop:
    - location: scripts/stop_container.sh
  AfterInstall:
    - location: scripts/start_container.sh
```

CodeDeploy Setup — EC2

- Create Application in CodeDeploy (compute platform: EC2)
- Install CodeDeploy Agent on EC2 instance
- Tag EC2 instance (e.g., Name: sample-python) — CodeDeploy uses tags to find targets

- Create IAM Role with CodeDeploy permissions, assign to EC2 instance
- Create Deployment Group: select EC2 instances by tag
- Create Deployment: point to GitHub repo + commit ID

Full CI/CD Pipeline

Source (GitHub) → CodeBuild (build + push Docker image to Docker Hub) → CodeDeploy (pull image + run container on EC2)

When code is pushed to GitHub main branch → CodePipeline triggers automatically → builds → deploys.

16. Secret Management

Three Options

Service	Best For	Key Feature
Systems Manager Parameter Store	Config values, non-sensitive strings, Docker registry URLs	Free tier available, IAM integrated
Secrets Manager	Passwords, DB credentials, API keys that need rotation	Automatic secret rotation, costs money
HashiCorp Vault	Multi-cloud, enterprise, on-prem + cloud hybrid	Not AWS-native, most flexible

When to Use Which

- Parameter Store: Docker registry URL, usernames, non-rotating config — it's FREE
- Secrets Manager: Database passwords, SSL certificates, anything with a rotation policy
- Cost optimization: Use Parameter Store where rotation isn't needed — Secrets Manager charges per secret per month

Secret Rotation

Secrets Manager can automatically rotate secrets on a schedule. For database credentials, it calls a Lambda function that generates a new password, updates the secret in Secrets Manager, and updates the database — all automatically.

Accessing from Code

- IAM Role on the service (Lambda, EC2, CodeBuild) must have permission to read the parameter/secret
- Services can retrieve values at runtime — secrets never hardcoded in code

17. AWS Config — Compliance Monitoring

AWS Config continuously monitors your AWS resources and evaluates them against rules to detect non-compliant configurations.

Core Concepts

- Compliant: Resource obeys the defined rule
- Non-Compliant: Resource violates the rule — generates a finding
- Config Rule: A check that evaluates resources (AWS managed or custom Lambda)
- Configuration Item: A point-in-time snapshot of a resource's configuration

Custom Lambda Rule — Detailed Monitoring Check

Example: Company rule says all EC2 instances must have detailed monitoring enabled. If a developer creates an EC2 without it:

- AWS Config detects the change, triggers the Lambda function
- Lambda checks if `instance['Monitoring']['State'] == 'enabled'`
- If not enabled → calls `config_client.put_evaluations()` with `NON_COMPLIANT` status
- AWS Config console shows the instance as non-compliant

Setup Steps

- Create Lambda function (Python) with the evaluation logic
- Set Lambda timeout to 10 seconds (default 3 is too short)
- Grant Lambda IAM permissions: `EC2FullAccess`, `AWS_ConfigRole`, `CloudWatchFullAccess`
- Go to AWS Config → Create custom Lambda rule → provide Lambda ARN
- Set Evaluation Mode to 'When configuration changes'
- Select EC2 Instance as the resource type to monitor

18. AWS Cloud Migration Strategy

A structured approach to moving workloads from on-premises to AWS.

The 5-Phase Framework

Phase	Name	Frequency
1	Preparation	One-time
2	Planning	One-time
3	Migrate	Repeats per phase
4	Monitor	Repeats per phase

Phase	Name	Frequency
5	Optimize	After all phases complete

The 7 Rs of Migration

Strategy	What Changes	When to Use
Rehost (Lift & Shift)	Where it runs — move as-is to EC2	Fast migration, no changes needed
Replatform (Lift, Tinker & Shift)	How it runs — use cloud features (multi-AZ, RDS)	Get cloud benefits without code changes
Refactor / Rearchitecture	Architecture — redesign as cloud-native microservices	Need capabilities current architecture can't deliver
Relocate	Infrastructure — move entire K8s cluster to EKS/ROSA	Already on K8s, want managed control plane
Retain	Nothing — stay on-prem	Regulatory, compliance, or dependency constraints
Retire	Decommission entirely	Unused, redundant, or outdated applications
Repurchase	Replace with SaaS (Salesforce, Workday, ServiceNow)	Your custom app does something a mature SaaS does better

❑ **Key Point:** Interview trap: 'You must break monolith into microservices before migrating.' FALSE. Only Refactor/Rearchitecture requires this. Rehost, Replatform, and Repurchase work fine with a monolith.

Migration Phases Planning

- Phase 1: Least critical services — low risk, proves the concept
- Phase 2: Moderate criticality
- Phase 3: Most critical (payments, auth, transactions) — migrate last when you have confidence

Database Migration — Critical Points

Databases require extra care — application downtime is acceptable, data loss is never acceptable.

- Choose the right managed service: MySQL → RDS for MySQL, PostgreSQL → RDS for PostgreSQL, high performance → Amazon Aurora
- Why RDS over EC2 self-managed: RDS handles backups, patching, replication, failover automatically
- Before migrating: Create a live synchronized backup (continuous replication — not a static snapshot)
- Never migrate to/from the production database directly — always to a new database
- Switch traffic only after thorough testing
- Keep backup running for a defined period after migration — rollback if issues appear

❑ **Note:** Rollback plan: If migration fails, the live-synced backup database is always current. Redirect traffic to it — zero data loss.

19. AWS CLI — Command Line Interface

Installation

```
curl https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip -o awscliv2.zip
unzip awscliv2.zip
sudo ./aws/install
```

Configuration

Create access keys: AWS Console → top-right username → Security Credentials → Access Keys

```
aws configure
# Prompts for: Access Key ID, Secret Access Key, Region, Output format
```

Common Commands

Command	Description
aws ec2 describe-instances	List all EC2 instances
aws s3 ls	List all S3 buckets
aws s3 cp file.txt s3://bucket/	Upload file to S3
aws iam list-users	List all IAM users
aws ecr get-login-password ...	Authenticate Docker to ECR
aws eks update-kubeconfig ...	Configure kubectl for EKS cluster
aws logs describe-log-groups	List CloudWatch log groups

20. Interview Quick Reference

Common Interview Questions & Answers

Question	Key Answer Points
What is the difference between SG and NACL?	SG = instance level, stateful, allow only. NACL = subnet level, stateless, allow & deny
What is a NAT Gateway?	Allows private subnet instances to make outbound internet calls without exposing them publicly

Question	Key Answer Points
Why use Fargate over EC2 for ECS/EKS?	No infrastructure management, auto-scales, pay per usage — but more expensive per unit
What is IRSA in EKS?	IAM Roles for Service Accounts — pods get temporary AWS credentials via OIDC, no access keys stored
Difference between ALB and NLB?	ALB = Layer 7 (HTTP-aware, path-based routing). NLB = Layer 4 (TCP/UDP, ultra-fast, static IP)
What are the 7 Rs of migration?	Rehost, Replatform, Refactor, Relocate, Retain, Retire, Repurchase
When to use Secrets Manager vs Parameter Store?	Secrets Manager for rotation (DB creds, certs). Parameter Store for config values (cheaper)
What is CloudFormation drift?	When actual resource config differs from the CFT template — detected via Stack → Detect Drift
What is an Ingress Controller in EKS?	A pod (AWS LB Controller) that watches Ingress resources and creates/configures the actual ALB
What is OIDC in EKS?	Identity provider that enables IRSA — allows K8s service accounts to assume IAM roles securely

Architecture Patterns

Highly Available Web Application

- 2+ AZs with public + private subnets in each
- Auto Scaling Group for EC2 in private subnets
- Application Load Balancer in public subnets
- RDS Multi-AZ for database
- Route 53 with health checks for DNS failover

Serverless Architecture

- API Gateway → Lambda → DynamoDB
- S3 + CloudFront for static frontend
- Cognito for authentication
- Secrets Manager for sensitive config

Container Architecture (EKS)

- EKS cluster with Fargate or managed node groups
- ECR for container images
- ALB Ingress Controller for traffic routing
- IRSA for pod-level IAM permissions
- CloudWatch Container Insights for monitoring

Cost Optimization Checklist

- Use Spot Instances for batch/fault-tolerant workloads
- Right-size instances based on CloudWatch CPU/memory metrics
- Delete unused EBS snapshots (Lambda automation)
- Use S3 Intelligent-Tiering or Glacier for infrequent data
- Reserved Instances or Savings Plans for stable workloads
- Use Parameter Store instead of Secrets Manager where rotation isn't needed
- Set S3 lifecycle policies to auto-delete old versions/objects

Good luck with your interview!

Practice these concepts hands-on — reading notes is not enough. Build it, break it, fix it.